

Segurança em Computação

Pipeline Seguro: SAST,
DAST, IAST, SCA e Secrets

Prof. Jean Everson Martina

Da vulnerabilidade ao pipeline seguro



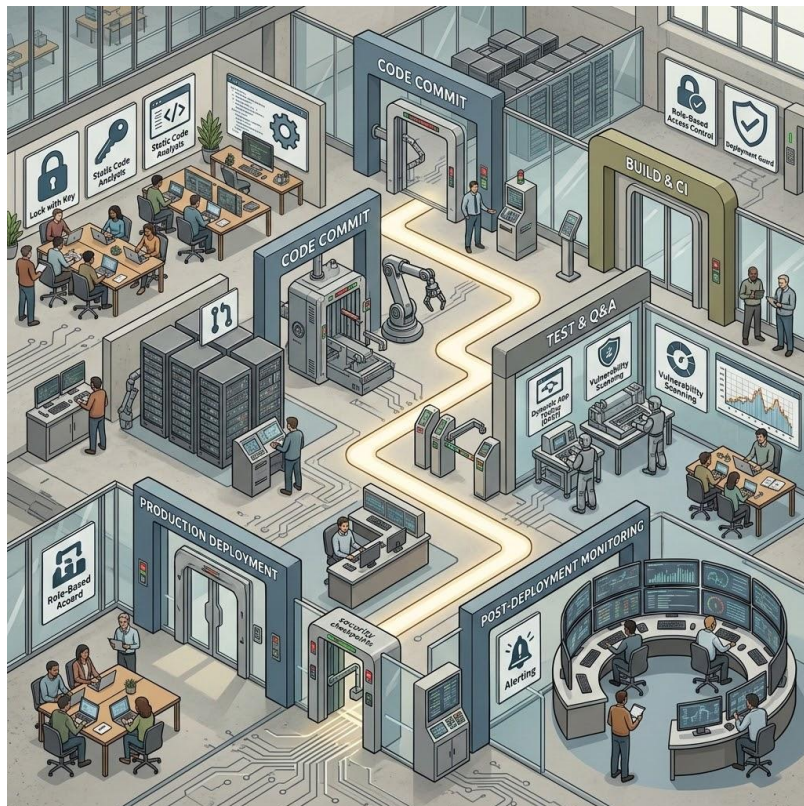
- Na aula anterior vimos como aplicações modernas falham em autorização, autenticação, validação, configuração e dependências.
- Muitas dessas falhas nascem antes da produção e poderiam ser detectadas durante o desenvolvimento.
- Outras só aparecem quando a aplicação está rodando em ambiente realista.
- O pipeline é o lugar onde segurança deixa de ser revisão tardia e vira validação contínua.
- A pergunta agora é: onde o processo deveria detectar cada falha antes que ela chegasse ao usuário?

O que é um pipeline CI/CD

- CI significa Continuous Integration: integrar código frequentemente, com testes e validações automáticas.
- CD pode significar Continuous Delivery ou Continuous Deployment, dependendo do grau de automação até produção.
- Um pipeline transforma código em artefato executável, imagem, pacote ou serviço implantado.
- Etapas comuns incluem commit, build, testes, análise, empacotamento, deploy e monitoramento.
- Cada etapa pode introduzir riscos ou detectar falhas antes que elas avancem.



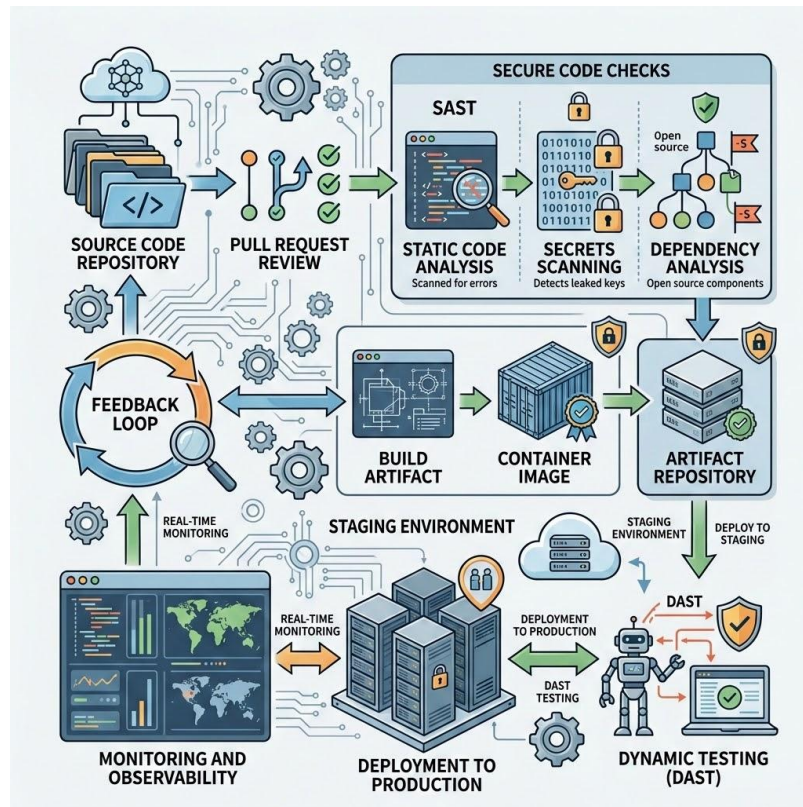
Onde a segurança entra no pipeline



- Antes do commit entram revisão, threat modeling leve e boas práticas locais de desenvolvimento.
- No commit e no pull request entram SAST, secrets scanning, linting seguro e revisão automatizada.
- No build entram SCA, análise de dependências, geração de artefatos e validação de imagens.
- Em staging entram DAST, IAST, testes funcionais e validação em runtime.
- No deploy e na produção entram policies, observabilidade, alertas e feedback para o backlog.

Mapa completo do pipeline seguro

- Código → Build → Testes → Análises → Artefato → Staging → Deploy → Monitoramento.
- SAST, secrets scanning e revisão automatizada atuam cedo, quando o custo de correção ainda é baixo.
- SCA, SBOM e análise de imagem ajudam a controlar dependências, pacotes e artefatos.
- DAST e IAST testam comportamento da aplicação em execução, antes ou durante validação em staging.
- Logs, métricas e alertas fecham o ciclo, conectando operação, resposta e melhoria contínua.



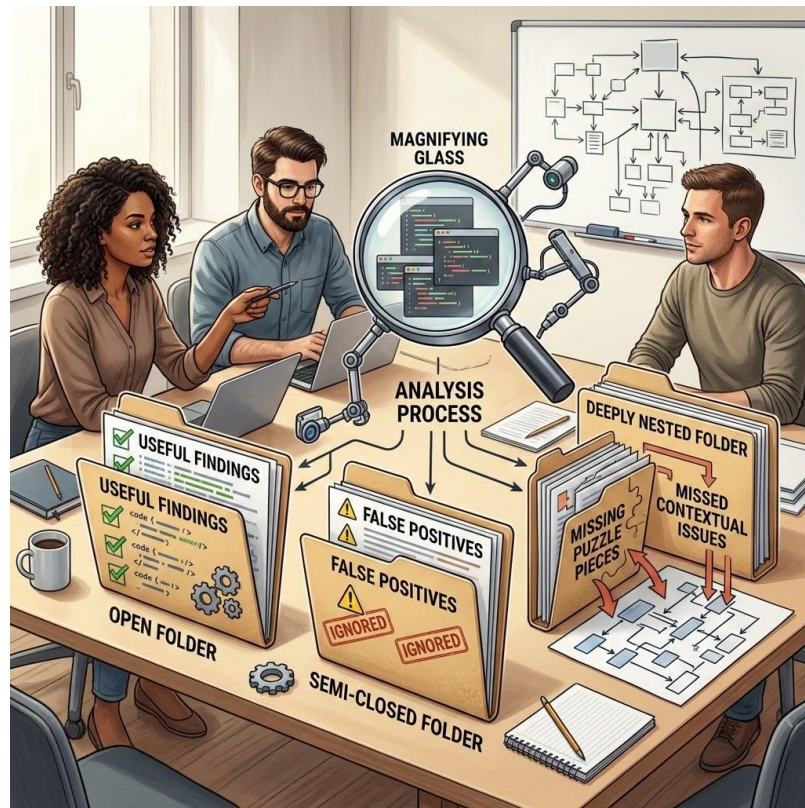
SAST: olhando o código antes da execução



- SAST significa Static Application Security Testing.
- Ele analisa código fonte, bytecode ou representação intermediária sem executar a aplicação.
- Busca padrões inseguros, uso perigoso de APIs, fluxos suspeitos e vulnerabilidades conhecidas no código.
- Pode rodar em commits, pull requests e pipelines de integração contínua.
- O maior valor do SAST é oferecer feedback cedo, quando a correção ainda é barata.

O que SAST encontra bem e onde ele falha

- SAST funciona bem para padrões conhecidos, chamadas perigosas e problemas repetitivos de implementação.
- É útil para detectar uso inseguro de APIs, concatenação suspeita, validação ausente e práticas proibidas.
- Pode gerar falsos positivos quando não entende contexto de negócio ou fluxo real da aplicação.
- Também pode gerar falsos negativos quando a vulnerabilidade depende de configuração, runtime ou integração externa.
- SAST ajuda muito, mas não substitui revisão de arquitetura, threat modeling e testes dinâmicos.



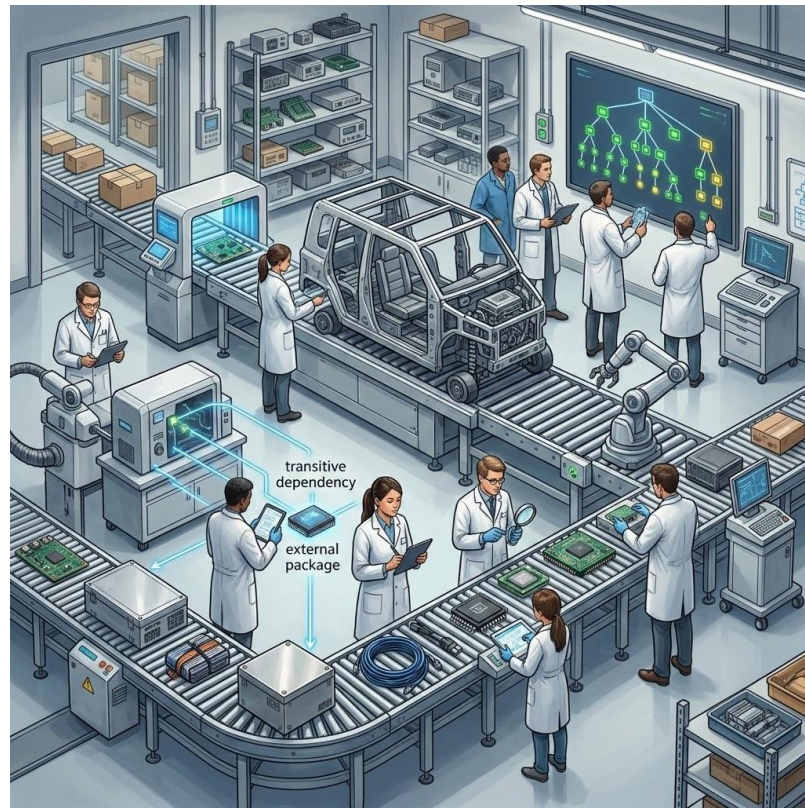
Secrets scanning: vazamento antes do deploy



- Secrets incluem senhas, tokens, chaves privadas, API keys, certificados e credenciais de cloud.
- Vazamentos acontecem em commits, arquivos .env, logs, imagens de container e configurações de pipeline.
- Um secret vazado pode dar acesso direto a banco, API, cloud ou ambiente de produção.
- Detectar o vazamento cedo é importante, mas apagar o commit não resolve sozinho.
- A resposta correta é revogar, rotacionar e investigar onde aquela credencial foi usada.

SCA: analisando dependências e componentes

- SCA significa Software Composition Analysis.
- Ele identifica bibliotecas, frameworks, pacotes, versões e vulnerabilidades conhecidas em componentes externos.
- Analisa dependências diretas e transitivas, muitas vezes invisíveis para o time de desenvolvimento.
- Usa bases como CVE, NVD e advisory databases para indicar riscos conhecidos.
- O objetivo é responder: estamos executando algum componente vulnerável ou abandonado?



SCA não é só lista de CVEs



- Nem toda vulnerabilidade conhecida é explorável no contexto real da aplicação.
- Uma dependência pode estar presente, mas não ser chamada por caminhos expostos.
- Atualizar uma biblioteca pode corrigir um risco, mas também quebrar compatibilidade ou comportamento.
- Priorização deve combinar severidade, exposição, explorabilidade, criticidade do ativo e facilidade de correção.
- O problema não é ter muitos alertas, mas não saber quais realmente importam.

SCA não é só lista de CVEs



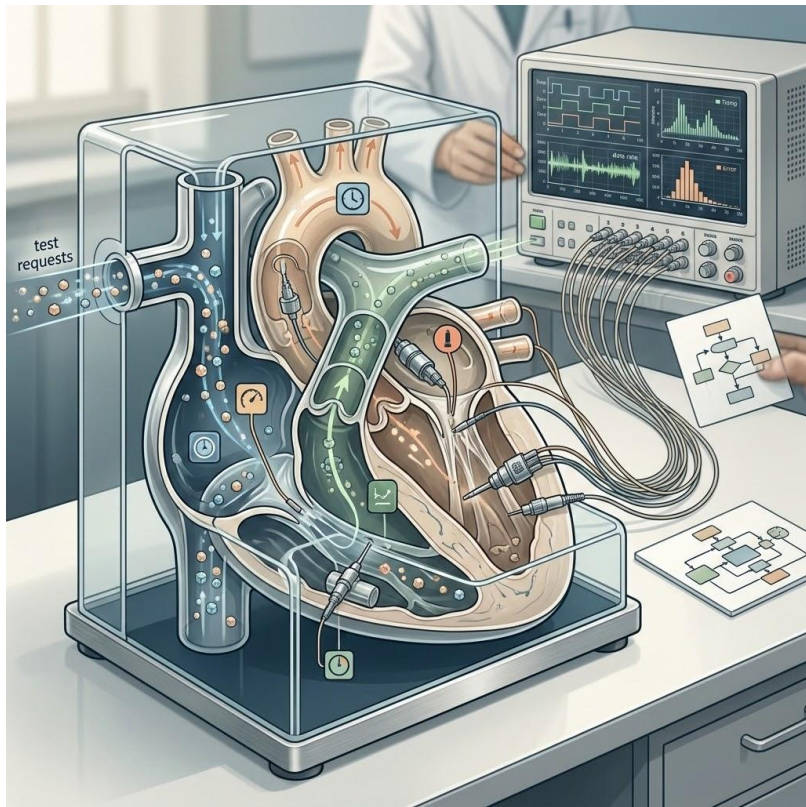
- Nem toda vulnerabilidade conhecida é explorável no contexto real da aplicação.
- Uma dependência pode estar presente, mas não ser chamada por caminhos expostos.
- Atualizar uma biblioteca pode corrigir um risco, mas também quebrar compatibilidade ou comportamento.
- Priorização deve combinar severidade, exposição, explorabilidade, criticidade do ativo e facilidade de correção.
- O problema não é ter muitos alertas, mas não saber quais realmente importam.

DAST no pipeline: útil, mas com cuidado

- DAST precisa que a aplicação esteja rodando e acessível em um ambiente de teste.
- Ele pode demorar mais que SAST e precisa de configuração adequada para não gerar ruído excessivo.
- Scans ativos podem alterar dados, acionar fluxos reais ou produzir efeitos indesejados se o ambiente for mal preparado.
- Em pipeline, DAST costuma funcionar bem como baseline scan em staging ou ambiente efêmero.
- DAST sem escopo e sem controle vira um usuário automatizado problemático.



IAST: sensores dentro da aplicação



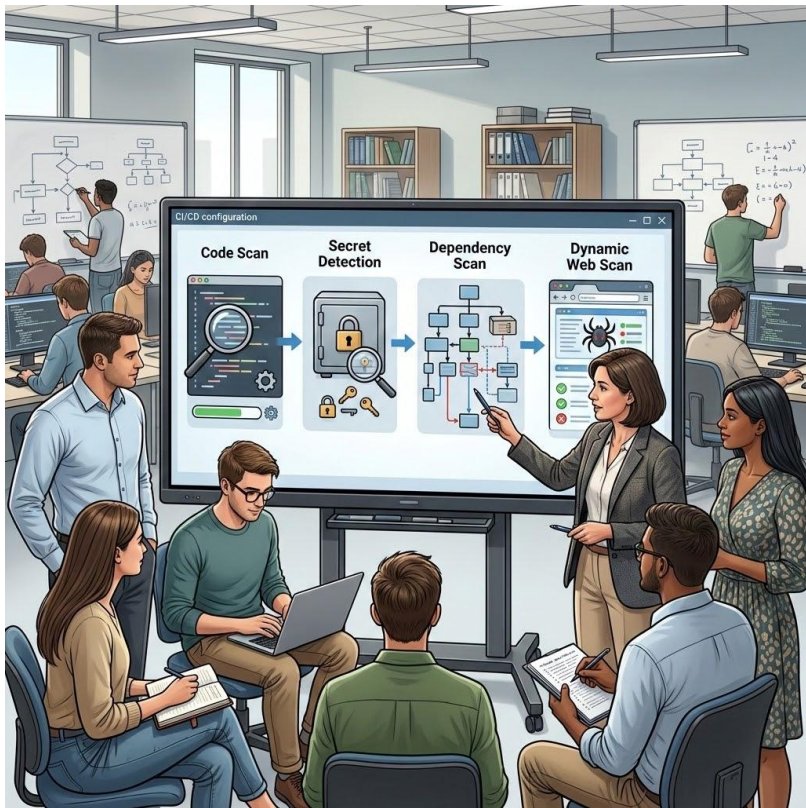
- IAST significa Interactive Application Security Testing.
- Ele instrumenta a aplicação durante a execução, usando agentes ou sensores no runtime.
- Combina testes em execução com visibilidade interna sobre fluxos de dados, chamadas e bibliotecas.
- Ajuda a reduzir falsos positivos porque observa se o caminho vulnerável realmente foi executado.
- É útil em ambientes de teste de integração, mas exige suporte da linguagem, framework e plataforma.

Comparando SAST, DAST, IAST, SCA e secrets

- SAST analisa o código antes da execução e encontra padrões inseguros cedo.
- DAST testa a aplicação rodando a partir da perspectiva externa de requisições e respostas.
- IAST observa a aplicação por dentro durante a execução e adiciona contexto de runtime.
- SCA analisa componentes externos, versões, dependências e vulnerabilidades conhecidas.
- Secrets scanning identifica credenciais expostas antes que elas virem acesso indevido.
- Essas técnicas não competem: elas respondem perguntas diferentes em momentos diferentes.



Exemplo simples de pipeline com validações



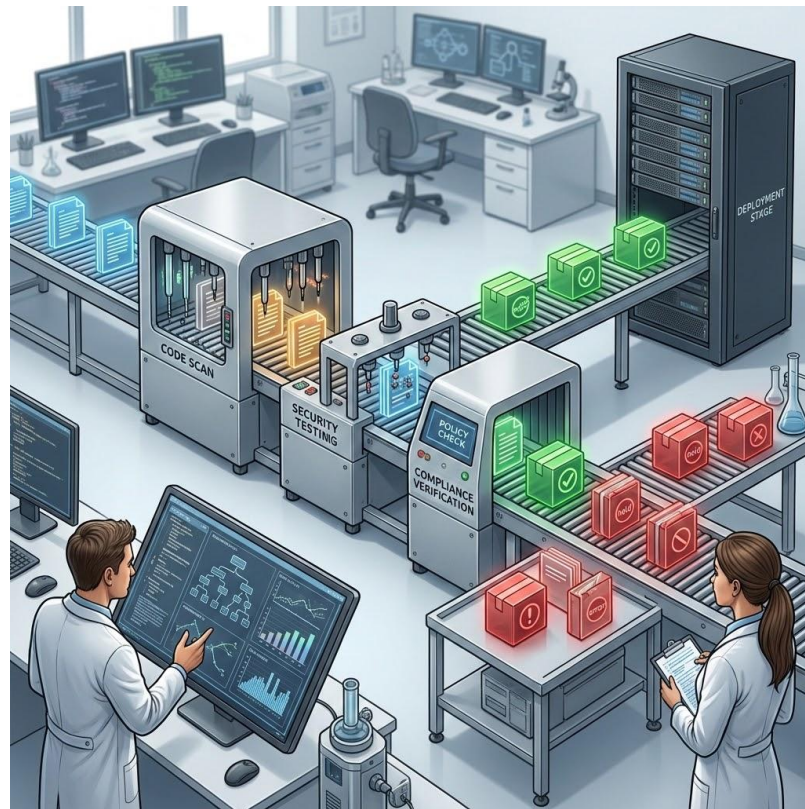
- Exemplo simplificado de etapas em um pipeline:

```
security:
  steps:
    - run: semgrep scan
    - run: gitleaks detect
    - run: trivy fs .
    - run: zap-baseline http://app-staging
```

- A ideia não é decorar comandos, mas entender a lógica de composição das validações.
- Cada ferramenta responde uma pergunta diferente sobre código, secrets, dependências e comportamento da aplicação.
- O resultado precisa virar decisão: bloquear, corrigir, aceitar risco ou registrar no backlog.

Quality gates: quando bloquear o pipeline

- Quality gates definem critérios mínimos para um código avançar no pipeline.
- Eles podem bloquear merge, build, deploy ou promoção para produção.
- Exemplos: secret detectado, CVE crítica explorável, falha alta no DAST ou política obrigatória violada.
- Gates precisam ser claros, justos, calibrados e compreendidos pelo time.
- Um gate bom protege; um gate ruim vira burocracia e todo mundo aprende a contornar.



Priorização: severidade não é risco



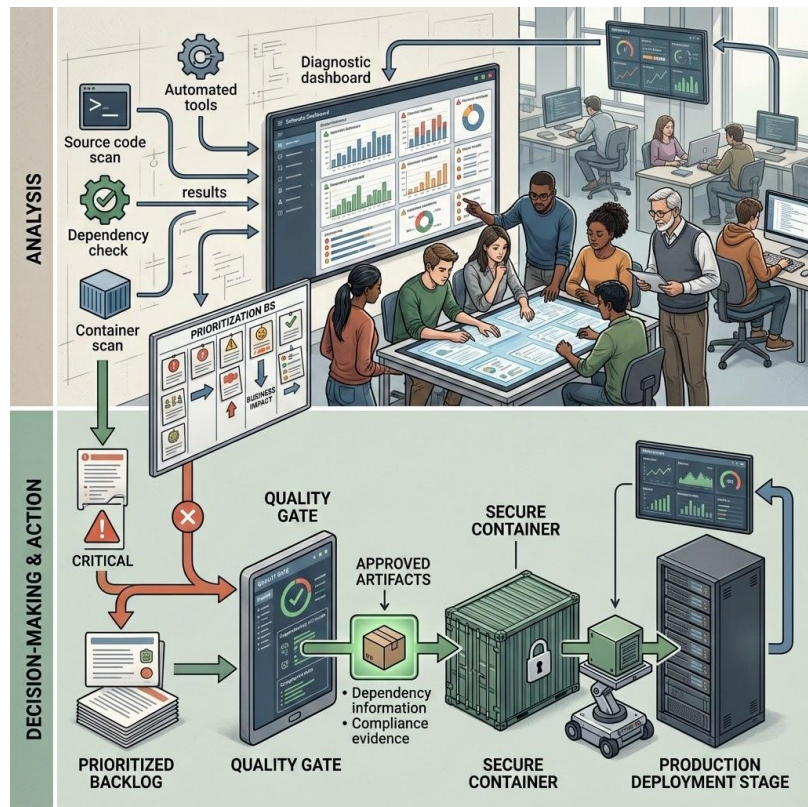
- Severidade mede gravidade técnica, mas não captura todo o contexto operacional.
- Risco combina severidade, exposição, explorabilidade, impacto e criticidade do ativo afetado.
- Uma falha crítica interna pode ser menos urgente que uma falha alta exposta publicamente.
- Priorização ruim gera filas intermináveis de alertas e pouca redução real de exposição.
- O pipeline deve alimentar backlog, métricas e decisões de engenharia, não apenas produzir relatórios.

Limites da automação

- Ferramentas automatizadas não entendem completamente o contexto de negócio da aplicação.
- Scanners podem gerar falsos positivos, falsos negativos e excesso de ruído operacional.
- Automação sem calibração cria fadiga, atrasos e rejeição cultural pelas equipes.
- Métricas ruins incentivam comportamento ruim, como ignorar alertas ou tratar segurança como checklist.
- DevSecOps exige automação, mas também exige julgamento, revisão e responsabilidade compartilhada.



Da detecção automática à engenharia segura



- Hoje vimos como segurança pode ser incorporada ao pipeline de desenvolvimento.
- SAST, DAST, IAST, SCA e secrets scanning respondem perguntas diferentes sobre riscos diferentes.
- Quality gates transformam achados técnicos em decisões operacionais.
- Priorização evita que a equipe se afogue em alertas e ignore o que realmente importa.
- A próxima aula aprofunda supply chain, SBOM, Infrastructure as Code e Policy as Code.
- DevSecOps não é rodar scanner: é transformar segurança em decisão contínua de engenharia.